🗡️ RS001 Week 12 — English Worksheet

Topic: Three Battles, Three Different Choices · **Course:** Text Adventure (Python) · **Time:** about 45 minutes

This week you think about how a whole battle fits inside one **function** called `fight()`, then how calling it three times gives three different enemies and three different action menus. One function, many battles — you will read it, predict it, and explain it in your own words.

> 🧠 Words to know: **function**, **call**, **parameter**, **return**, **reuse**

```
def fight(enemy_name, actions):
    game.say(f"=== {enemy_name} appears! ===")
    game.say(f"Choices: {', '.join(actions)}")
    action = game.ask("Do what? ").strip().lower()
    if action == actions[0]:
        game.say("A hard-fought win!")
        return True
    return False

fight("forest goblin", ["fight", "trap", "run"])
```

1 · Predict 🔮

Read each piece of code. Run it in your head and write what you think happens.

```
actions = ["fight", "trap", "run"]
game.say(", ".join(actions))
```

`join` glues the list together. What single line is printed?

```
def fight(enemy):
    game.say(f"{enemy} appears!")
    return True

won = fight("orc")
game.say(won)
```

Two things print. What are they?

```
def fight(enemy, actions):
    game.say(f"{enemy}: {actions[0]}")

fight("slime", ["fire", "spell"])
fight("knight", ["sneak", "talk"])
```

The same function is called twice. What two lines appear on screen?

2 · Spot the Bug 🐛

Each block was meant to do something, but it is broken. Read what it is **supposed** to do, fix it, then explain why the original was wrong.

Bug A — `fight()` should tell the caller whether the player won by returning a value. Right now it prints but returns nothing, so `won` is empty.

```
def fight(enemy):
    game.say(f"You beat the {enemy}!")

won = fight("goblin")
game.say(f"Victory: {won}")
```

Hint: add a `return True` at the end of the function.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug B — Each call should use a **different** action menu. Right now both battles share the exact same actions, even though they pass different lists.

```
def fight(enemy, actions):
    game.say(f"{enemy}: {', '.join(['fight', 'run'])}")

fight("slime", ["fire", "spell", "run"])
fight("knight", ["sneak", "talk", "run"])
```

Hint: the function should use its `actions` parameter, not a hard-coded list.

Write the fixed code:

Why was it wrong? Why does your fix work?

Bug C — `', '.join(actions)` needs a **list** of words to glue. Here a single string was passed, so the menu prints letter by letter.

```
def fight(enemy, actions):
    game.say(f"{enemy}: {', '.join(actions)}")

fight("bat", "fly")
```

Hint: wrap the action(s) in a list: `["fly"]`.

Write the fixed code:

Why was it wrong? Why does your fix work?

3 · Explain the Code 📖

Read this working battle carefully. It is one `fight()` function called three times for three different enemies.

```
def fight(enemy_name, actions):
    game.say(f"=== {enemy_name} appears! ===")
    game.say(f"Choices: {', '.join(actions)}")
    action = game.ask("Do what? ").strip().lower()
    if action == actions[0]:
        game.say("A hard-fought win!")
        return True
    return False

fight("forest goblin", ["fight", "trap", "run"])
fight("cave troll", ["smash", "dodge", "run"])
fight("ice wraith", ["freeze", "burn", "run"])
```

What are the two parameters of `fight()`, and what does each one hold?

The function is written once but the battle happens three times. How does that work?

When does `fight()` return `True`, and when does it return `False`?

Each call passes a different `actions` list. Where in the function does that list get used?

Why is using one `fight()` function better than copy-pasting the battle code three times?

4 · Explain Your Lesson Code 🎥

In today's live lesson you wrote your own `fight()` code. Now explain it. Take a short video on your phone (or a parent's phone) showing your code — you may let it run — and teach it like the viewer has never coded. Try to use these words: **function**, **call**, **parameter**, **return**, **reuse**.

> 1. Show the one `fight()` function you wrote in the lesson.
> 2. Point to its parameters and say what each one holds.
> 3. Show the three different calls and read out their different menus.
> 4. Explain why one function you reuse is better than three copies.

Write what you will say in your video. Plan it here before you record.

Submit ✅

Send this worksheet + a video explaining your lesson code to teacher on KakaoTalk.